

Elucidating the Theoretical Underpinnings of Surrogate Gradient Learning in Spiking Neural Networks

Julia Gygax

julia.gygax@fmi.ch

Friedemann Zenke

friedemann.zenke@fmi.ch

*Friedrich Miescher Institute for Biomedical Research and Faculty of Science,
University of Basel, Basel 4056, Switzerland*

Training spiking neural networks to approximate universal functions is essential for studying information processing in the brain and for neuromorphic computing. Yet the binary nature of spikes poses a challenge for direct gradient-based training. Surrogate gradients have been empirically successful in circumventing this problem, but their theoretical foundation remains elusive. Here, we investigate the relation of surrogate gradients to two theoretically well-founded approaches. On the one hand, we consider smoothed probabilistic models, which, due to the lack of support for automatic differentiation, are impractical for training multilayer spiking neural networks but provide derivatives equivalent to surrogate gradients for single neurons. On the other hand, we investigate stochastic automatic differentiation, which is compatible with discrete randomness but has not yet been used to train spiking neural networks. We find that the latter gives surrogate gradients a theoretical basis in stochastic spiking neural networks, where the surrogate derivative matches the derivative of the neuronal escape noise function. This finding supports the effectiveness of surrogate gradients in practice and suggests their suitability for stochastic spiking neural networks. However, surrogate gradients are generally not gradients of a surrogate loss despite their relation to stochastic automatic differentiation. Nevertheless, we empirically confirm the effectiveness of surrogate gradients in stochastic multilayer spiking neural networks and discuss their relation to deterministic networks as a special case. Our work gives theoretical support to surrogate gradients and the choice of a suitable surrogate derivative in stochastic spiking neural networks.

Friedemann Zenke is the corresponding author.

1 Introduction

Our brains efficiently process information in spiking neural networks (SNNs) that communicate through short stereotypical electrical pulses called spikes. SNNs are an indispensable tool for understanding information processing in the brain and instantiating similar capabilities in silico. Like conventional artificial neural networks (ANNs), SNNs require training to implement specific functions. However, SNN models, which are commonly simulated in discrete time (Eshraghian et al., 2023), are not differentiable due to the binary nature of the spike, which precludes the use of standard gradient-based training techniques based on backpropagation (BP; Rumelhart et al., 1986). There are several ways to overcome this problem. One can dispense with hidden layers altogether (Gütig & Sompolinsky, 2006; Memmesheimer et al., 2014), but this limits the network's expressivity and precludes solving more complex tasks. Alternatively, one makes the neuron model differentiable (Huh & Sejnowski, 2018) or considers the timing of existing spikes for which gradients exist (Bohte et al., 2002; Mostafa, 2017; Wunderlich & Pehle, 2021; Klos & Memmesheimer, 2023). This approach, however, requires additional mechanisms to create or remove spikes, as well as dealing with silent neurons, where the spike time is not defined. Finally, one can replace the gradient with a suitable surrogate (Bohte, 2011; Courbariaux et al., 2016; Esser et al., 2016; Zenke & Ganguli, 2018; Bellec et al., 2018; Neftci et al., 2019; Eshraghian et al., 2023). In this article, we focus on the latter. Surrogate gradient (SG) approaches are empirically successful and not limited to changing the timing of existing spikes while working with nondifferentiable neuron models.

However, at present surrogate gradients are a heuristic that lacks a theoretical foundation. Consequently, we still do not understand why SG descent works well in practice (Zenke & Vogels, 2021). We do not know if it is optimal in any sense, nor do we know if there are better strategies for updating the weights. To address this gap in our understanding we analyze the commonalities and differences of SGs with theoretically well-founded approaches based on stochastic networks. Specifically, we focus on smoothed probabilistic models (SPMs; Neftci et al., 2019; Jang et al., 2019) and the more recently proposed stochastic automatic differentiation (stochAD) framework (Arya et al., 2022). SPMs typically rely on stochasticity to smooth out the optimization landscape in expectation to enable gradient computation on this smoothed loss. In this article, we particularly focus on neuron models with escape noise (Gerstner et al., 2014), which are commonly used to smooth the nondifferentiable spikes in expectation and for which exact gradients are computable (Pfister et al., 2006; Brea et al., 2013). However, extending SPMs to multilayer neural networks has been difficult because they preclude using BP and, hence, efficient gradient estimation requires additional approximations (Gardner et al., 2015). In contrast, stochAD is a recently developed framework for automatic

differentiation (AD) in programs with discrete randomness, that is, with discrete random variables. While this opens the door for BP or other recursive gradient computation schemes, the framework has not yet been applied to SNNs.

Here, we jointly analyze the above methods, elucidate their relations, provide a rigorous theoretical foundation for SG-descent in stochastic SNNs, and discuss their implications for deterministic SNNs. We first provide some background information on each of the above methods before starting our analysis with the case of a single perceptron. From there, we move to the more complex case of multilayer perceptrons (MLPs), where we elaborate the theoretical connection between SGs and stochAD and end with multilayer networks of leaky integrate-and-fire (LIF) neurons. Finally, we substantiate the theoretical results with empirical simulations.

2 Background on Surrogate Gradients, Smoothed Probabilistic Models, and Stochastic Automatic Differentiation

Before analyzing the relation between SGs, gradients of the log-likelihood in SPMs, and stochAD in section 3, we briefly review these methods here. While their common goal is to compute reasonable weight updates, the first two approaches are specifically designed for training SNNs. The third approach aims to compute unbiased gradients of arbitrary functions with discrete randomness based on AD.

2.1 Smoothed Probabilistic Models. SPMs are based on stochastic networks in which the gradients are well defined in expectation (Neftci et al., 2019; Jang et al., 2019). Typically, such networks consist of a noisy neuron model, such as the LIF neuron with escape noise (Gerstner et al., 2014), and a probabilistic loss function. On the one hand, this includes models that require optimizing the log-likelihood of the target spike train being generated by the current model (Pfister et al., 2006). However, this method only works effectively without hidden units. On the other hand, SPMs also comprise models that follow a variational strategy (Brea et al., 2013), which makes them applicable to networks with a single hidden layer. In general, the gradients that are computed within the SPM framework are given by

$$\frac{\partial}{\partial w} \mathbb{E}[\mathcal{L}(w)],$$

where $\mathcal{L}$ is the loss and w is an arbitrary model parameter. The computational cost associated with evaluating the expected value of the loss $\mathbb{E}[\mathcal{L}]$ precludes training multilayer networks in practice. This is because SPMs lack support for AD, as we explain in detail in section 3.2.1.

2.2 Surrogate Gradients. SGs are a heuristic that relies on a continuous relaxation of the nondifferentiable spiking activation function that occurs

when computing gradients in SNNs (Neftci et al., 2019) and is commonly applied in deterministic networks. To that end, one systematically replaces the derivative of the hard threshold by a surrogate derivative (SD), also called pseudo-derivative (Bellec et al., 2018), when applying the chain rule. The result then serves as a surrogate for the gradient. For example, when computing the derivative of the time-discretized spike train S with respect to a weight w , the problematic derivative of a Heaviside $\partial H(u - \theta)/\partial u$ is replaced by an SD,

$$\frac{\partial \tilde{S}}{\partial w} \leftarrow \underbrace{\frac{\partial \sigma_{\beta_{\text{SG}}}(u - \theta)}{\partial u}}_{\text{SD}} \cdot \frac{\partial u}{\partial w}, \quad (2.1)$$

where the tilde denotes the introduction of the surrogate. $H(\cdot)$ denotes the Heaviside function, u is the membrane potential, θ is the firing threshold, and $\sigma_{\beta_{\text{SG}}}$ is a differentiable function parameterized by β_{SG} used to define the SD. Different functions are used in practice. For instance, the derivative of rectified linear functions or the arctangent function have been used successfully as SDs (Bohte et al., 2002; Esser et al., 2016; Bellec et al., 2018; Hammouamri et al., 2023), whereas SuperSpike (Zenke & Ganguli, 2018) used a scaled derivative of a fast sigmoid $h(x) = \frac{1}{(\beta_{\text{SG}}|x|+1)^2}$ with β_{SG} controlling its steepness. However, SNN training is robust to the choice of the SD (Zenke & Vogels, 2021; Herranz-Celotti & Rouat, 2024).

2.3 Stochastic Surrogate Gradients. In practice, SGs are most commonly used for training deterministic SNNs and have rarely been used in stochastic SNNs, with some notable exceptions (Gardner et al., 2015). Here we define stochastic SG descent analogous to the deterministic case as a continuous relaxation of the nondifferentiable spiking activation function applied to networks of stochastic neurons with escape noise (Gerstner & Kistler, 2002; Gerstner et al., 2014). In these networks, spike generation is stochastic in the forward path, whereby an escape noise function yields the neuronal firing probability. Equivalently, one can formulate stochastic neurons as neurons with a stochastic threshold Θ (Gerstner & Kistler, 2002). For the backward path, one considers the derivative of a differentiable function of the membrane potential. Analogous to the deterministic case, the derivative of a time-discretized spike train is hence given as

$$\frac{\partial \tilde{S}}{\partial w} \leftarrow \underbrace{\frac{\partial \sigma_{\beta_{\text{SG}}}(u - \bar{\Theta})}{\partial u}}_{\text{SD}} \cdot \frac{\partial u}{\partial w}, \quad (2.2)$$

where the mean threshold $\bar{\Theta}$ is equivalent to θ in the deterministic case (for details see section 8.2). In stochastic binary neural networks, the above

notion of SGs is known as the straight-through estimator (STE; Hinton, 2012; Bengio et al., 2013; Courbariaux et al., 2016; Yin et al., 2018).

2.4 Stochastic Automatic Differentiation. To compute derivatives efficiently, popular AD algorithms rely on the chain rule. The required terms can be calculated in forward or backward mode, with the latter being called BP (Marschall et al., 2019). To extend this to exact AD in programs with discrete randomness, which otherwise preclude analytical gradient computation, the *stochAD* framework (Arya et al., 2022) introduces stochastic derivatives. This framework provides an unbiased and composable solution to compute derivatives in discrete stochastic systems that does not suffer from the high variance found in methods that require sampling and averaging such as finite differences methods (see Figure 3 and section 3.2.2 for a comparison). To deal with discrete randomness, stochastic derivatives consider not only infinitesimally small continuous changes but also finite changes with infinitesimally small probability. As defined by Arya et al. (2022), a stochastic derivative of a random variable $X(p)$ consists of a triple (δ, w, Y) . Here δ is the “almost sure” derivative, meaning the derivative of the continuous part; w is the derivative of the probability of a finite change; and Y is the alternate value—the value of $X(p)$ in case of a finite jump. Although stochastic derivatives are suitable for forward mode AD, they cannot be used directly for BP as of now. This is because the application of the chain rule would require the derivatives to be scalars, whereas they consist of a triple. The *stochAD* framework outlines a way to convert them to a single scalar value, which Arya et al. (2022) call the “smoothed stochastic derivative.” This transformation renders the framework compatible with BP, albeit at the cost of introducing bias. Given a stochastic derivative, its smoothed stochastic derivative is

$$\tilde{\delta} = \mathbb{E}[\delta + w(Y - X(p)) | X(p)] \quad (2.3)$$

for one realization of the random variable $X(p)$. Arya et al. (2022) also showed that the smoothed stochastic derivatives recover the STE (Hinton, 2012; Bengio et al., 2013) for a Bernoulli random variable in their example A.8.

3 Analysis of the Relation between SGs, SPMs, and *stochAD* for Direct Training of SNNs

To fathom the theoretical foundation of SG learning, we focused on the theoretically well-grounded SPMs and the recently proposed *stochAD* framework. Because both approaches assume stochasticity whereas SGs are typically applied in deterministic settings, we focus our analysis on stochastic networks (see section 2). We later discuss deterministic networks as a special case.

To keep our analysis general and independent of the choice of the loss function $\mathcal{L}$, we consider the Jacobian $\nabla_w y$ defined at the network's output y where w are the trainable parameters or weights. For simplicity and without loss of generality, we consider networks with only one output such that the above is equivalent to studying

$$\nabla y = \left(\frac{\partial}{\partial w_1} y, \dots, \frac{\partial}{\partial w_n} y \right).$$

To further ease the analysis, we start with binary perceptrons, thereby neglecting all temporal dynamics and the reset of conventional spiking neuron models, while retaining the essential binary spike generation process (see Figure 1A and section 8.2). We begin our comparison by examining a single neuron before moving on to MLPs. We defer the discussion of LIF neurons to section 5.

3.1 Analysis of the Binary Perceptron. Before we compare the different methods of computing gradients or SGs, we define our deterministic and stochastic perceptron here.

3.1.1 Deterministic Perceptron. The deterministic perceptron is defined as

$$u = W^T x + b, \quad (3.1)$$

$$y = H(u - \theta), \quad (3.2)$$

where $H(\cdot)$ is again the Heaviside step function and u is the membrane potential, which depends on the weights W , the bias b , the input x , and the firing threshold θ . The Jacobian is related to the gradient via

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial w_i},$$

where the problematic derivative of the nondifferentiable Heaviside function appears in $\frac{\partial y}{\partial w_i}$ with w_i being the i th weight. When computing SGs, the derivative of the Heaviside function is replaced with the corresponding SD,

$$\frac{\partial}{\partial w_i} y(u - \theta) \leftarrow \frac{\partial}{\partial w_i} \sigma_{\beta_{\text{SG}}}(u - \theta). \quad (3.3)$$

Here we chose the SD as the derivative of a sigmoid with steepness β_{SG} , hence $\sigma_{\beta_{\text{SG}}}(u - \theta) = \frac{1}{1 + \exp(-\beta_{\text{SG}}(u - \theta))}$.

3.1.2 Stochastic Perceptron. To see how the above expression compares to SDs and derivatives of the expected output in SPMs in the corresponding

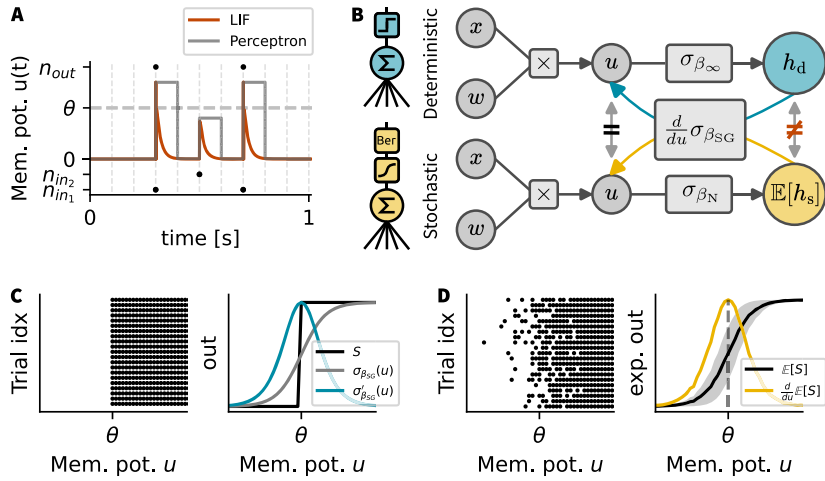


Figure 1: SDs are equivalent to derivatives of expected outputs in SPMs and smoothed stochastic derivatives in binary perceptrons. (A) Membrane potential dynamics of an LIF neuron (maroon) in comparison to the perceptron. When input spikes (from input neurons n_{in_1} and n_{in_2}) are received, they excite the LIF neuron, which causes the membrane potential to increase. Once it reaches the threshold, output spikes n_{out} are emitted. In the limit of a large simulation time step ($dt \gg \tau_{mem}$) and appropriate scaling of the input currents, the LIF neuron approximates a perceptron receiving time-locked input (gray line). (B) Left: The simplified computational graph of a deterministic (blue) and a stochastic (yellow) perceptron. Right: Forward pass in a deterministic (top) or stochastic (bottom) perceptron. The colored arrows indicate that both use the derivative of $\sigma_{\beta_{SG}}(\cdot)$ on the backward pass, which is the derivative of the expected output of the stochastic neuron in case $\beta_{SG} = \beta_N$. In the case of the deterministic neuron, this constitutes the SG used instead of the nonexisting derivative of the step function. (C) Left: Network output over multiple trials in the deterministic perceptron. Right: The SD (blue) is the derivative of a sigmoid (gray), which is used to approximate the nonexisting derivative of the step function (black). (D) Same as panel C but for the stochastic perceptron. Left: Escape noise leads to variability in the spike trains over trials. Right: The expected output follows a sigmoid, and we can compute the derivative (yellow) of the expected output.

stochastic setting, we consider the stochastic perceptron:

$$\begin{aligned}
 u &= W^T x + b, \\
 p &= f(u - \theta) = \sigma_{\beta_N}(u - \theta), \\
 y &\sim \text{Ber}(p).
 \end{aligned} \tag{3.4}$$

Note that the membrane potential u is equivalent to the deterministic case (see equation 3.1) as the models share the same input. To model stochasticity, we add escape noise. This means the perceptron fires with a probability p , which is a function f of the membrane potential. To that end, we choose $f(\cdot)$ as the sigmoid function $\sigma_{\beta_N}(u - \theta) = \frac{1}{1 + \exp(-\beta_N(u - \theta))}$, where β_N controls the steepness of the escape noise function (Brea et al., 2013). Importantly, there is some degree of freedom as to which escape noise function we choose. Other common choices in the realm of spiking neurons are the exponential or error function (Gerstner et al., 2014). For our current choice, the sigmoid, we find that it approaches the step function in the limit $\beta_N \rightarrow \infty$, and the stochastic perceptron becomes deterministic.

We now compare SG descent in a deterministic and a stochastic unit. To do so, we first incorporate the spike generation of the stochastic binary perceptron into the threshold and reformulate the stochastic firing using $H(u - \Theta)$. The stochastic threshold Θ is given by $\Theta = \theta + \xi$, where ξ is randomly drawn noise from a zero-mean distribution. This formulation is equivalent to stochastic firing as in equation 3.4 for a suitable choice of the distribution underlying ξ (see equation 8.5). The stochastic SD is then as in equation 2.2. Since the input and the weights are the same in both cases, both units have the same membrane potential. Because the SD depends only on the membrane potential, the resulting weight updates are equivalent provided that both use the same β_{SG} (see equations 2.1 and 2.2). Thus, the resulting SDs in the stochastic and deterministic perceptron are equivalent, although the outputs of the two units are generally different (see Figures 1C and 1D), and hence the overall weight update is different.

We now turn to comparing SGs to SPMs. In SPMs, the idea is to compute the derivative of the expected loss at the output to smooth out the discrete spiking nonlinearity (Pfister et al., 2006). However, as before, we consider the expected network output instead to make our analysis independent of the choice of loss function. In our setup, this amounts to computing the expected output of the stochastic perceptron,

$$\mathbb{E}[y] = \sigma_{\beta_N}(u - \theta),$$

and taking its derivative,

$$\frac{\partial}{\partial w_i} \mathbb{E}[y] = \frac{\partial}{\partial w_i} \sigma_{\beta_N}(u - \theta). \quad (3.5)$$

We note that the right-hand sides of expressions 3.5 and 3.3 are the same when setting $\beta_N = \beta_{SG}$. Thus, the derivative of the expected output of the stochastic Perceptron is equivalent to the SD of the output of the perceptron, if the SD is chosen such that it matches the derivative of the neuronal escape noise (see Figure 1B).

Finally, we compare the above findings to the derivative obtained from the stochAD framework. To that end, we first apply the chain rule given one realization of y ,

$$\frac{\partial y}{\partial w_i} = \frac{\partial}{\partial p} \text{Ber}(p) \frac{\partial}{\partial w_i} p, \quad (3.6)$$

and use the smoothed stochastic derivative of a Bernoulli random variable following the steps of Arya et al. (2022). According to their example A.8 (Arya et al., 2022), the right stochastic derivative for a Bernoulli random variable is given by $(\delta_R, w_R, Y_R) = (0, \frac{1}{1-p}, 1)$ if the outcome of the random variable was zero ($\text{Ber}(p) = 0$) and zero otherwise. The left stochastic derivative is given by $(\delta_L, w_L, Y_L) = (0, \frac{-1}{p}, 0)$ if $\text{Ber}(p) = 1$ and also zero otherwise. The corresponding smoothed versions are $\tilde{\delta}_R = \frac{1}{1-p} \cdot \mathbf{1}_{X(p)=0}$ and $\tilde{\delta}_L = \frac{1}{p} \cdot \mathbf{1}_{X(p)=1}$ (see Arya et al., 2022, or equation 2.3). Since every affine combination of the left and right derivatives is a valid derivative, we can use $(1-p)\tilde{\delta}_R + p\tilde{\delta}_L = 1$ as the smoothed stochastic derivative of the Bernoulli random variable. Hence, as previously seen in Arya et al. (2022), the smoothed stochastic derivative of a Bernoulli can be equal to 1. This results in

$$\frac{\partial y}{\partial w_i} \leftarrow 1 \cdot \frac{\partial}{\partial w_i} \sigma_{\beta_N}(u - \theta)$$

when inserted into equation 3.6. Yet again, we obtain the same right-hand side as above. It is worth noting that in the comparison with SG descent, we consider not only the smoothed stochastic derivative of the Bernoulli random variable but also its composition with the derivative of the firing probability, or equivalently, the neuron's escape noise function. This composition is necessary because we require a surrogate for the ill-defined derivative of the time-discretized spike train with respect to the membrane potential $\frac{\partial S}{\partial u}$ that is also valid in the deterministic case.

In summary, when analyzing different ways of computing or approximating derivatives, the resulting expressions are identical in single units if the SD matches the derivative of the escape noise function of the stochastic neuron model. In the single-neuron case, this match is equivalent to probabilistic smoothing of the loss, a well-known fact in the literature (Pfister et al., 2006), which allows computing exact gradients of the expected value. Nevertheless, the resulting weight updates are generally different in the deterministic and stochastic cases because these expressions also depend on the neuronal output, which is different in the deterministic and stochastic neurons. We will see in the next section which equivalences are broken in multilayer networks due to these and other notable differences.

3.2 Analysis of the Multilayer Perceptron. To analyze the relation of the different methods in the multilayer setting, we begin by examining

SPMs, which lack support for AD and thus an efficient algorithm to compute gradients. We then further discuss how stochAD provides smooth stochastic derivatives equivalent to SGs in multilayer networks.

3.2.1 Output Smoothing in Multilayer SPMs Precludes Efficient Gradient Computation. SPMs lack support for AD because they smooth the expected loss landscape through stochasticity and therefore require the calculation of expected values at the network output. While output smoothing allows the implementation of the finite difference algorithm, this algorithm does not scale to large models and is therefore of little practical use for training ANNs (Werfel et al., 2004; Lillicrap et al., 2020). The application of AD, however, requires differentiable models, like standard ANNs, so that the chain rule can be used to decompose the gradient computation into simple primitives. This composability is the basis for efficient recursive algorithms like BP and real-time recurrent learning (RTRL; Marschall et al., 2019).

To see why SPMs do not support AD, we consider a simple example network: Let y be the output of a binary neural network with input x and two hidden layers with activities h_1, h_2 (see Figure 2). The output y has the firing probability $p_y = \sigma_{\beta_N}(w_y^T h_2)$ as in equation 3.4 and the hidden layers have the firing probabilities $p_1 = \sigma_{\beta_N}(w_1^T x)$ and $p_2 = \sigma_{\beta_N}(w_2^T h_1)$, respectively. We are looking for a closed-form expression of the derivative of the expected output for each parameter, for example, $\frac{\partial}{\partial w_1} \mathbb{E}[y]$ for weight w_1 (see Figure 2B). In a deterministic and differentiable network, one can use the chain rule to split the expression into a product of partial derivatives as $\frac{\partial}{\partial w_1} y = \frac{\partial y}{\partial p_y} \frac{\partial p_y}{\partial h_2} \frac{\partial h_2}{\partial p_2} \dots \frac{\partial p_1}{\partial w_1}$ (see Figure 2A). However, this is not possible for SPMs because $\frac{\partial}{\partial w_1} \mathbb{E}[y] \neq \mathbb{E} \left[\frac{\partial}{\partial w_1} y \right]$, where the right-hand side would involve the derivative of the nondifferentiable binary output.

Even if we could exchange the expectation value and the derivative, we would still be faced with the fact that the expectation of a product is usually not equal to the product of expectation values unless the factors are independent (see Figure 2C), hence

$$\begin{aligned} & \mathbb{E} \left[\frac{\partial}{\partial p_y} y \frac{\partial}{\partial h_2} p_y \frac{\partial}{\partial p_2} h_2 \dots \frac{\partial}{\partial w_1} p_1 \right] \\ & \neq \mathbb{E} \left[\frac{\partial}{\partial p_y} y \right] \mathbb{E} \left[\frac{\partial}{\partial h_2} p_y \right] \mathbb{E} \left[\frac{\partial}{\partial p_2} h_2 \right] \dots \mathbb{E} \left[\frac{\partial}{\partial w_1} p_1 \right] \\ & \neq \frac{\partial}{\partial p_y} \mathbb{E}[y] \frac{\partial}{\partial h_2} \mathbb{E}[p_y] \frac{\partial}{\partial p_2} \mathbb{E}[h_2] \dots \frac{\partial}{\partial w_1} \mathbb{E}[p_1]. \end{aligned}$$

Clearly, in neural networks, the factors are not independent because the activity of downstream neurons depends on the activity of their upstream partners. Thus, it is not obvious how to compute gradients in multilayer

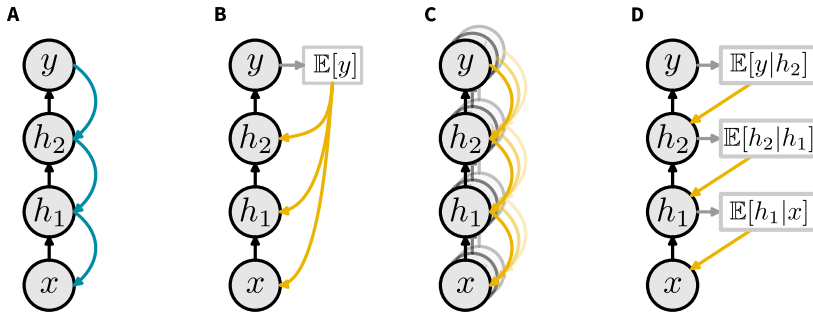


Figure 2: Derivative computation in MLPs. Schematic of an example network for which (surrogate) derivatives are computed according to different methods. The colored arrows indicate where partial derivatives are calculated. (A) SG descent relies on the chain rule for efficient gradient computation in a deterministic MLP. Thus, the derivative of the output with respect to a given weight is factorized into its primitives, which are indicated by the colored arrows. (B) SPMs approach the problem of nondifferentiable spike trains by adding noise and then smoothing the output based on its expected value. Since this method does not allow the use of the chain rule, the derivative for each weight must be computed directly. (C) The derivative and the expected value are not interchangeable, which makes this option mathematically invalid. Furthermore, it is not possible to achieve the necessary smoothing using the expected value after such an interchange. (D) Smoothed stochastic derivatives in stochAD use the expected value of each node to compute the derivative. However, the method relies on expectation values conditioned on the activity of a specific forward pass.

SPM networks. We will see that stochAD suggests sensible solutions to circumvent the problems mentioned above and makes the use of AD possible in stochastic networks. Consequently, in the following, we consider only SGs and stochAD, which support BP.

3.2.2 Stochastic Automatic Differentiation Bestows Theoretical Support to Surrogate Gradients. Here we show how smoothed stochastic derivatives for stochastic binary MLPs relate to SGs. Arya et al. (2022) defined the smoothed stochastic derivative (repeated in equation 2.3 for convenience) for one realization of a random variable $X(p)$, where the discrete random variables are usually Bernoulli random variables in stochastic binary MLPs.

To gain an intuitive understanding of the essence of smoothed stochastic derivatives, we consider a single realization of a stochastic program, which, in our case, is a stochastic MLP. In other words, we run a single stochastic forward pass in the MLP and condition on its activity. At each point in time, each neuron either emits a one or a zero, that is, a spike or none. Thus, we can think of all these binary decisions as edges in a tree, with

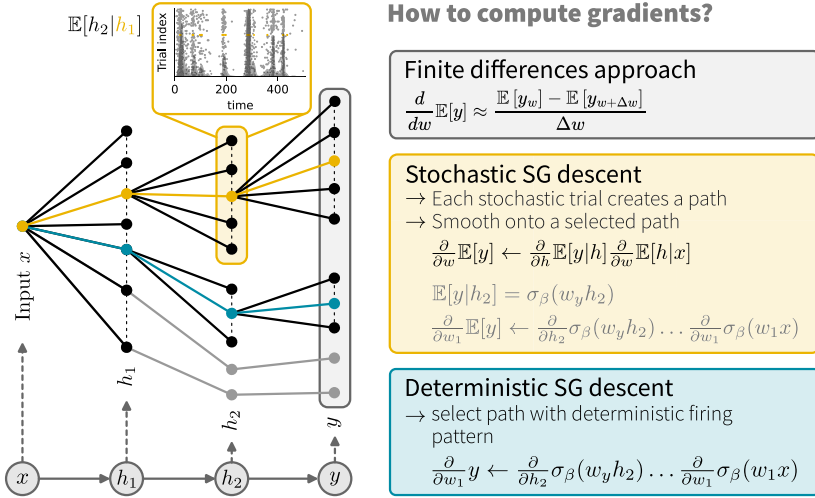


Figure 3: SGs correspond to smoothed stochastic derivatives in stochastic SNNs. The tree illustrates the discrete decisions associated with the binary spike generation process at different points over different layers of a stochastic MLP. A single forward pass in the network corresponds to a specific path through the tree, which yields a specific set of spike trains. Another forward pass will result in a different path and spike patterns. Computing gradients using finite differences requires randomly sampling paths from the network and evaluating their averaged loss before and after a given weight perturbation. Although this approach is unbiased for small perturbations, the random path selection results in high variance. Furthermore, that approach is not scalable to large networks. Stochastic SG descent is equivalent to smoothed stochastic derivatives in the stochAD framework. To compute the gradient, we roll out the network once and sample a random path in the tree, which we now keep fixed (yellow). At each node, we compute the expected output given the fixed activation of the previous layer $\mathbb{E}[h_i|h_{i-1}]$, which yields a low-variance estimate (see the inset: spike raster, selected trial shown in yellow, spike trains of other trials in gray, expectation shown as shaded overlay). By choosing a surrogate function that matches the escape noise process, both methods give the same derivative for a spike with respect to the membrane potential. Deterministic SG descent can be seen as a special case in which the random sampling of the path is replaced by a point estimate given by the deterministic rollout (blue).

each node representing the spike pattern of all units in a given layer. Any rollout of a forward pass then corresponds to a particular randomly chosen path through this tree (see Figure 3). These paths originate from the same root for a given input x , yet the stochasticity in all of them is independent. In this tree of possible spike patterns, it is easy to understand how different methods for gradient estimation work.

We first consider the finite differences method. In this approach, the loss is evaluated once using the parameters w and once using $w + \Delta w$, either considering a single trial or averaging over several independent trials. Hence, one randomly samples paths in the tree of spike patterns and compares their averaged output before and after weight perturbation. Since the randomness of the sampled paths is “uncoupled,” the finite difference approach results in high variance (see Figure 3, gray), which scales with the inverse of the squared perturbation Δw . However, smaller perturbations allow a more accurate gradient estimation (Glasserman, 2003; Fu, 2006). Reducing this variance requires instead the introduction of a coupling between the sampled paths, which is at the heart of stochAD (Arya et al., 2022). The key idea is to condition on the previous layer’s output h_{i-1} and then consider all possible spike patterns for the current layer, $\mathbb{E}[h_i|h_{i-1}]$. Thus, we can consider the expected layer activity, given a randomly chosen path, for example, the yellow path in Figure 3. For this path, the derivatives now need to be smoothed at each node. To do so, we compute the expectation in each layer conditioned on the activity of the previous layer along the selected path. After smoothing, it is possible to compute the path-wise derivative along a selected path with activity h_1^*, h_2^*, y^* :

$$\frac{\partial}{\partial w_1} \mathbb{E}[y] = \frac{1}{n_{\text{paths}}} \sum_{\text{paths}} \frac{\partial}{\partial p_y} \mathbb{E}[y|h_2^*] \frac{\partial}{\partial h_2} \mathbb{E}[p_y|h_2^*] \frac{\partial}{\partial p_2} \mathbb{E}[h_2|h_1^*] \cdots \frac{\partial}{\partial w_1} \mathbb{E}[p_1|x].$$

Here we averaged over all possible paths, that is, all possible combinations of the activities h_1^*, h_2^* . In practice, it is rarely possible to average over all possible combinations, and one instead uses a Monte Carlo estimate, which still yields significantly lower variance than other schemes and can be computed efficiently using BP for a single path per update.

Given the above method for computing smoothed stochastic derivatives, we are now in the position to understand their relationship to SGs. Since we condition on a specific path in the tree of all possible spike patterns, we only compute derivatives of the expected output h_i conditional on the output of the previous layer h_{i-1} according to the chosen path at each node. Such an approach exactly corresponds to treating each node as a single unit as in section 3.1. As we saw above, the derivative of the expected output of a single unit with escape noise is equivalent to the corresponding smoothed stochastic derivative and the SD in that unit both with or without escape noise. Furthermore, when using SGs, there is no difference in how the method is applied in single-versus multilayer cases and there is always a well-defined path to condition on. So SGs can also be understood as treating all units at each layer as single units. The same is true for smoothed stochastic derivatives in the stochAD framework. Thus, the stochAD framework provides theoretical support to SGs in stochastic networks and also stipulates that the SD should be matched to mirror the derivative of the neuronal escape

noise, thereby extending this matching principle from the single neuron case (Pfister et al., 2006; Brea et al., 2013) to the multilayer network setting and removing some of the ambiguity associated with choosing this function in practice (Zenke & Vogels, 2021).

3.2.3 Surrogate Gradients in Deterministic Networks. SGs are commonly used to train deterministic SNNs. Deterministic networks show two essential differences to their stochastic counterparts, which result in different weight updates from what we have learned above. First, while a stochastic network typically selects a different path in the tree of spike patterns in each trial, there is only one path per input and parameter set in the deterministic case. This means that the expectation value over sampling random paths is replaced by a point estimate provided by the deterministic rollout. This difference introduces a selection bias in the estimation of the stochastic implementation. However, in practice, this bias is often small (see supplementary Figure S1). Second, when training deterministic networks with SGs, the slope of the SD at threshold (β_{SG}) is typically chosen ad hoc. Typical values are on the order one compared to the neuronal rheobase. This is stark in contrast to the “correct” slope of the corresponding asymptotic escape noise function from taking the limit $\beta_N \rightarrow \infty$. Approaching this limit empirically with β_{SG} leads to unstable training and poor performance (Zenke & Vogels, 2021). The choice $\beta_{SG} \neq \beta_N$ inevitably results in different weight updates. While this is a desired effect that allows training in the first place by avoiding the problem of zero and infinite weight updates, it is less clear whether this approximation has any undesirable off-target consequences. To address this question, we examine the properties of SGs in deterministic networks in the next section.

4 Analysis of Surrogate Gradient Properties in Deterministic Networks

By design, SGs deviate from the actual gradient, as they provide a nonzero surrogate in cases where the actual gradient is zero. It is not clear a priori what consequences such a deviation has and whether SGs are gradients at all, which can be obtained by differentiating a surrogate loss. However, it is difficult to get a quantitative understanding when comparing either to the zero vector or to the ill-defined derivative at the point of the spike. To take a closer look at how SGs deviate from actual gradients and the resulting consequences, we now move to differentiable network models that have a well-defined gradient. While SG training is not required in such networks, it allows us to develop a quantitative understanding of the commonalities and differences we should expect. This comparison also allows us to check whether SGs satisfy the formal criteria of gradients.

4.1 Deviations of SGs from Actual Gradients in Differentiable MLPs and the Consequences.

We first sought to understand whether SGs point

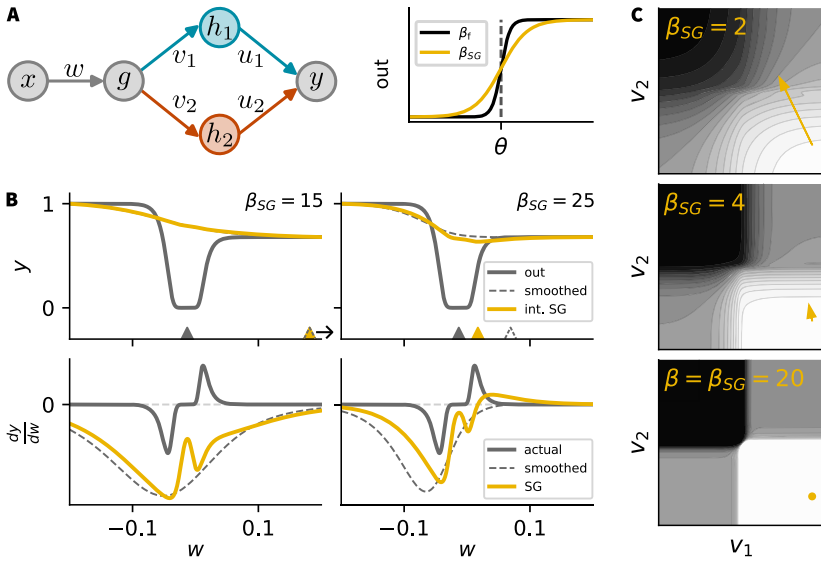


Figure 4: SGs deviate from actual gradients in differentiable MLPs. (A) Schematic of a differentiable network (left) with sigmoid activations (right) for which we compute an SD using the derivative of a flatter sigmoid (yellow) in contrast to the actual activation (black). (B) Top row: Network output (solid gray), smoothed network output (dashed), and integrated SD (yellow) as a function of w . The triangles on the x -axis indicate the minimum of the corresponding curves. Bottom row: Derivatives of the top row. Left and right correspond to a flatter ($\beta_{SG} = 15$) and a steeper ($\beta_{SG} = 25$) SD. See Table 1 for network parameters. Note that the actual derivative and the surrogate can have opposite signs. (C) Heat map of the optimization landscape along v_1 and v_2 for different β_{SG} values (top to bottom). While the actual gradient can be asymptotically zero (see yellow dot, bottom), the SD provides a descent direction (yellow arrow), thereby enabling learning (top and middle).

in a “similar direction” as the actual gradient. Specifically, we asked whether SGs ensure sign concordance, that is, whether they preserve the sign of the gradient components. To investigate this question, we consider a small network with input x and output y (see Figure 4A) defined by

$$\begin{aligned}
 g &= \sigma_{\beta_f}(wx), \\
 h_1 &= \sigma_{\beta_f}(v_1g), \\
 h_2 &= \sigma_{\beta_f}(v_2g), \\
 y &= \sigma_{\beta_f}(u_1h_1 + u_2h_2).
 \end{aligned} \tag{4.1}$$

Table 1: Parameter Values for Sign Flip Example.

Parameter	w	v_1	v_2	u_1	u_2	β_f	β_{SG}	Input x
Value	0	0.05	0.1	1	-1	100	25	1

Notes: Parameter values for the network in Figure 4A, with input $x = 1$, which serve as an example, that SGs can have the opposite sign of the actual gradient and thus point toward the opposite direction. Therefore, we cannot guarantee the SG to align with the actual gradient.

The network parameters are denoted by w , v_1 , v_2 , u_1 , and u_2 , while g , h_1 , and h_2 are the hidden layer activations. This network has a well-defined gradient and provides a minimal working example. To study the effect of computing an SD, we replace the sigmoid parameterized with β_f used in the forward pass by a surrogate function with β_{SG} , which is used to compute the SD in the backward pass. Hence the SD for a steeper sigmoid is given by

$$\frac{\tilde{\partial}}{\partial w} \sigma(\beta_f \cdot wx) \leftarrow x \cdot \beta_{SG} \cdot \sigma(\beta_{SG} \cdot wx) \cdot (1 - \sigma(\beta_{SG} \cdot wx)) \quad (4.2)$$

with $\beta_f > \beta_{SG}$. As before, the deterministic binary perceptron corresponds to the limiting case $\beta_f \rightarrow \infty$.

To investigate the differences between the SG and the actual gradient, we are particularly interested in the derivative of the output with respect to the hidden layer weight w (see Figure 4A). The partial derivative of the output with respect to w is given as

$$\frac{\partial}{\partial w} y = \underbrace{\frac{\partial y}{\partial h_1} \frac{\partial h_1}{\partial g} \frac{\partial g}{\partial w}}_{\text{blue path}} + \underbrace{\frac{\partial y}{\partial h_2} \frac{\partial h_2}{\partial g} \frac{\partial g}{\partial w}}_{\text{maroon path}}.$$

Now inserting the SDs using equation 4.2 leads to

$$\begin{aligned} \frac{\tilde{\partial}}{\partial w} y &= \left(\frac{\tilde{\partial} y}{\partial h_1} \frac{\tilde{\partial} h_1}{\partial g} + \frac{\tilde{\partial} y}{\partial h_2} \frac{\tilde{\partial} h_2}{\partial g} \right) \frac{\tilde{\partial} g}{\partial w} \\ &\leftarrow \underbrace{(u_1 v_1 \cdot \sigma'_{\beta_{SG}}(v_1 g) + u_2 v_2 \cdot \sigma'_{\beta_{SG}}(v_2 g))}_{\text{responsible for sign flip}} \\ &\quad \cdot \underbrace{\beta_{SG}^3 \cdot \sigma'_{\beta_{SG}}(u_1 h_1 + u_2 h_2) \cdot \sigma'_{\beta_{SG}}(wx)}_{\text{positive factor}} \cdot x, \end{aligned} \quad (4.3)$$

where only the first factor in equation 4.3 which consists of two terms, determines the relative sign of the SD with respect to the actual derivative. This is because the second factor and β_{SG} are always positive. Furthermore, the derivative of the sigmoid is always positive independent of the choice of β_i or β_{SG} . Finally, x , the input data, does not change its sign dependent on β_{SG} . However, the first factor can change its sign, since it is a summation of two nonlinear functions with changed hyperparameters β_i or β_{SG} and different weights, which may be negative. For instance, when we use specific parameter values (given in Table 1) in equation 4.3, the SD has the opposite sign of the actual derivative (see Figure 4B). Thus, already in this simple example, there is no guarantee that the sign of the SG is preserved with respect to the actual gradient. As a consequence, following the SG will not necessarily find parameter combinations that correspond to a minimum of the loss.

4.2 Surrogate Gradients Are Not Gradients of a Surrogate Loss. Given the above insight, we wondered whether an SG can be understood as the gradient of a surrogate loss that is not explicitly defined. To answer this question, we note that if, and only if, the SG is the gradient of a scalar function (i.e., corresponds to a conservative field), then integrating over any closed path must yield a zero integral. To check this, we considered the approximate Jacobian obtained using the SDs in the above example and numerically computed the integral over a closed circular path parameterized by the angle α in a two-dimensional plane in parameter space for different values of β_{SG} ,

$$I_{SG} = \int_{0^\circ}^{360^\circ} \frac{\tilde{dy}(\theta_\alpha)}{d\theta_\alpha} \frac{d\theta_\alpha}{d\alpha} d\alpha,$$

where the network parameters at any position are given by θ_α (see Figures 5A and 5B and section 8.3). We found that integrating the SD did not yield a zero integral, whereas using the actual derivatives resulted in a zero integral as expected. Importantly, this difference was not explained by numerical integration errors due to the finite step size (see Figure 5C). Thus, SGs cannot be understood as gradients of a surrogate loss.

5 From Perceptrons to LIF Neurons

In our above treatment, we focused on binary perceptrons for ease of analysis. In the following, we show that our findings readily generalize to networks of LIF neurons. To that end, we consider LIF neurons in discrete time that share many commonalities with the binary perceptron (see also Figure 1A). To illustrate these similarities, let us consider a single LIF neuron

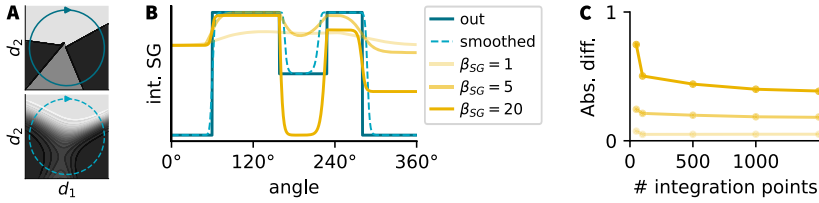


Figure 5: SGs are not gradients. (A) Heat map of the network output while moving along two random directions in the parameter space of the example network with step activation (top) and sigmoid activation (bottom; see Figure 4A). The circles indicate a closed integration path through parameter space, starting at the arrowhead. (B) Integral values of SDs as a function of the angle along the closed circular path shown in panel A. Different shades of yellow correspond to different values of β_{SG} . The blue lines correspond to the actual output of the network with step activation function (solid) or sigmoid activation (dashed). The integrated actual derivative of the network with sigmoid activation matches the output (dashed line) and is thus not visible in the plot. (C) Absolute difference between actual loss value and integrated SD as a function of the number of integration steps. The numerical integrals converge to finite values. Thus the observed difference is not an artifact of the numerical integration.

with index i described by the following discrete-time dynamics:

$$I_i[n+1] = \lambda_s I_i[n] + \sum_j w_{ij} S_j[n], \quad (5.1)$$

$$U_i[n+1] = (\lambda_m U_i[n] + (1 - \lambda_m) I_i[n]) (1 - S_i[n]), \quad (5.2)$$

$$S_i[n] = H(U_i[n] - \theta), \quad (5.3)$$

where w_{ij} describes the synaptic weights between the neuron i and the input neuron j , $\lambda_s = \exp\left(-\frac{\Delta t}{\tau_s}\right)$ and $\lambda_m = \exp\left(-\frac{\Delta t}{\tau_m}\right)$, where Δt is the time step, τ_s is the synaptic time constant, and τ_m is the membrane time constant. While the first two equations characterize the linear temporal dynamics of the synaptic current I and membrane potential U , the last equation captures the nonlinearity of the neuron, its spiking output S . Thus, in this formulation, we can think of an LIF neuron as a binary perceptron whose inputs are first processed through a linear filter cascade (i.e., the neuronal dynamics) and additionally have a reset mechanism given by the last factor in equation 5.2.

In the stochastic case, this filter does not change. In fact, we keep the same equations for the synaptic current, equation 5.1, and the membrane potential, equation 5.2. However, instead of a deterministic Heaviside function as in equation 5.3, we use a stochastic spike generation mechanism with escape noise, which is independent for each neuron i given its membrane

potential:

$$p_i[n] = \sigma_{\beta_N}(U_i[n] - \theta), \quad (5.4)$$

$$S_i[n] \sim \text{Ber}(p_i[n]). \quad (5.5)$$

Again, this mechanism is in direct equivalence with the stochastic perception case (see equation 3.4) and permissive to computing smoothed stochastic derivatives.

The derivative of the current and the membrane potential with respect to the weights induces their own dynamics:

$$\begin{aligned} \frac{d}{dw_{ij}} I_i[n+1] &= \lambda_s \frac{d}{dw_{ij}} I_i[n] + S_j[n], \\ \frac{d}{dw_{ij}} U_i[n+1] &= \lambda_m (1 - S_i[n]) \frac{d}{dw_{ij}} U_i[n] \\ &\quad + (1 - \lambda_m)(1 - S_i[n]) \frac{d}{dw_{ij}} I_i[n] \\ &\quad - (\lambda_m U_i[n] + (1 - \lambda_m) I_i[n]) \frac{d}{dw_{ij}} S_i[n], \end{aligned} \quad (5.6)$$

where we used the product rule to include the derivative of the reset term. To compute the smoothed stochastic derivative of the time-discretized spike train, we use the affine combination of the left and right smoothed stochastic derivatives of a Bernoulli random variable according to Arya et al. (2022) to get

$$\begin{aligned} \frac{d}{dw_{ij}} S_i[n] &\leftarrow \underbrace{\frac{d}{dp_i[n]} \text{Ber}(p_i[n])}_{=1} \frac{d}{dU_i[n]} \sigma_{\beta_N}(U_i[n]) \frac{d}{dw_{ij}} U_i[n] \\ &= \underbrace{\frac{d}{dU_i[n]} \sigma_{\beta_N}(U_i[n])}_{\text{SD}} \frac{d}{dw_{ij}} U_i[n]. \end{aligned} \quad (5.7)$$

Again, we find that this exactly recovers SGs as introduced in Zenke and Ganguli (2018) for $\beta_{\text{SG}} = \beta_N$ when conditioning on the deterministic spiking path and, hence, confirms the equality between SGs and smoothed stochastic derivatives. Thus, our analysis establishes a link between the escape noise function of stochastic LIF neurons and the shape of the SD in multilayer SNNs. A similar connection between escape noise and exact derivatives in expectation was previously described in shallow SNNs (Pfister et al., 2006; Brea et al., 2013).

Going further, we can use equation 5.7 to write equation 5.6 such that it only depends on current and membrane potential derivatives,

$$\frac{d}{dw_{ij}} U_i[n+1] = \left(\lambda_m (1 - S_i[n]) - \underbrace{\sigma'_{\beta_N}(U_i[n]) \cdot (\lambda_m U_i[n] + (1 - \lambda_m) I_i[n])}_{\text{derivative through reset}} \right) \cdot \frac{dU_i[n]}{dw_{ij}} + (1 - \lambda_m) (1 - S_i[n]) \frac{dI_i[n]}{dw_{ij}}, \quad (5.8)$$

and can be computed forward in time. Most SNN simulators avoid BP through the reset term (Zenke & Ganguli, 2018; Eshraghian et al., 2023) as this empirically improves performance for poorly scaled SGs (Zenke & Vogels, 2021). Therefore, it is common practice to set the right-hand term in the parentheses in equation 5.8, that is, the derivative through the reset term, to zero. Conversely, strict adherence to stochAD would suggest keeping the reset term when backpropagating. However, similar to Zenke and Vogels (2021), we find no difference in performance whether we backpropagate through the reset or not when the SG is scaled with $\frac{1}{\beta_{SG}}$ (see supplementary Figure S2), while without this scaling, BP through the reset negatively affects performance. Overall, we have shown that our results obtained from the analysis of perceptrons are transferable to SNNs and hence confirm again the equality between SGs and smoothed stochastic derivatives in the stochAD framework.

6 Surrogate Gradients Are Ideally Suited for Training Stochastic SNNs

We have seen that SG descent is theoretically justified by stochAD, albeit better supported for stochastic spiking neuron models. This finding also suggests that SG descent is suitable for training stochastic SNNs, with deterministic SNNs being a special case (see Figure 3). Next, we wanted to confirm this insight numerically. To that end, we trained deterministic and stochastic SNNs with SG descent on a deterministic spike train matching and a classification task.

For the spike train matching task, we assumed 200 input neurons with frozen Poisson spike trains. We then set up a feedforward SNN with one hidden layer, initialized in the fluctuation-driven regime (Rossbroich et al., 2022). We used supervised SG training in a deterministic and a stochastic version of the same network to match 200 target spike trains that were given by a dithered picture of the Radcliffe Camera (see Figure 6A and section 8.4.1). Both networks learned the task, albeit with visibly different hidden-layer activity (see Figure 6B). The deterministic version outperformed the stochastic SNN in terms of L_2 -distance at 1 ms, the temporal resolution of the simulation. However, when comparing their

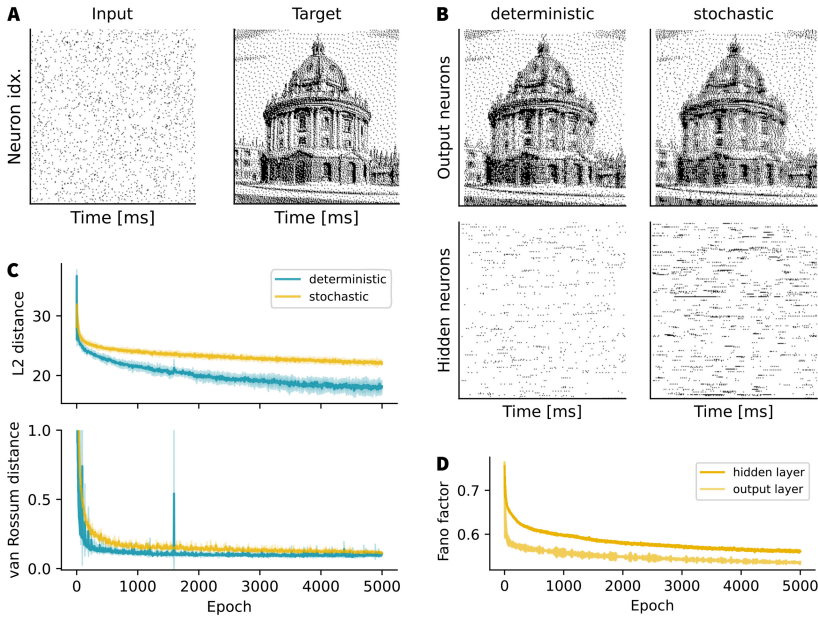


Figure 6: SGs successfully train stochastic SNNs on a spike train matching task. (A) Spike raster plots of the input and target spike trains of the spike train matching task. Time is shown on the x -axis, the neuron indices are on the y -axis, and each dot is a spike. The task is to convert the given frozen Poisson input spike pattern (left) into a structured output spike pattern depicting the Radcliffe Camera in Oxford (right). (B) Top: Output spike raster plots after training of the deterministic (left) and the stochastic SNNs (right). Although both methods faithfully reproduce the overall target structure, the deterministic network is slightly better at matching the exact timing. Bottom: Hidden-layer spike raster plots. Despite the similar output, the two networks show visibly different hidden-layer activity patterns. (C) L_2 loss (top) and van Rossum distance (bottom) throughout training for the two network models. While the deterministic network outperforms the stochastic one in terms of L_2 distance, the difference is negligible for the van Rossum distance. (D) Average Fano factor throughout training in the hidden and output layer of the stochastic network. Although the stochastic network reduces its variability during training to match the deterministic target, its hidden layer still displays substantial variability at the end of training.

outputs according to the van Rossum distance (van Rossum, 2001) with an alpha-shaped filter kernel equivalent to the ϵ -kernel of the LIF neuron ($\tau_{\text{mem}} = 10$ ms, $\tau_{\text{syn}} = 5$ ms), we found no difference in loss between the stochastic and deterministic networks (see Figure 6C). Finally, we observed that the Fano factor, a measure of stochasticity, of the stochastic SNN

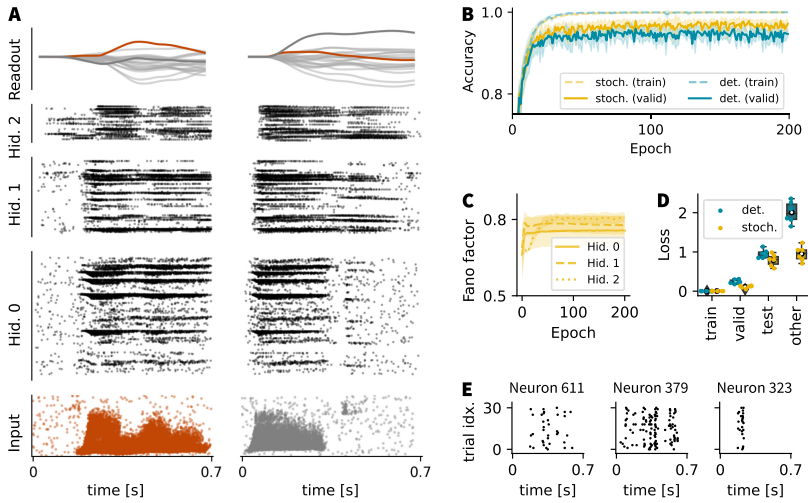


Figure 7: SGs can successfully train stochastic ConvSNNs. (A) Snapshot of network activity for two correctly classified sample inputs from the SHD data set. Top: Readout unit activity over time, the colored line indicates the activity of the unit that corresponds to the correct class. Below: Spike raster plots of the three convolutional hidden layers. The spike raster plots of the inputs are shown at the bottom in red and gray. Time is on the x -axis, the neuron index is on the y -axis, and each dot represents a spike. (B) Learning curves for training (dashed) and validation (solid) accuracy for the stochastic and the deterministic case (average over $n = 6$ trials $\pm$ std). (C) The mean Fano factor of the different layers in the stochastic network throughout training ($\pm$ std, $n = 3$). (D) The first three pairs of boxes show train, validation, and test loss of the ConvSNN as in panel A for the stochastic and the deterministic case for $n = 6$ random initializations. The right-most boxes show test loss for the opposite activation function. This means the network trained deterministically is tested with stochastic activation and vice versa. (E) Raster plots over trials of the spiking activity of three randomly chosen units from the second hidden layer. The units show clear trial-to-trial variability reminiscent of cortical activity.

dropped during training to better accommodate the deterministic target (see Figure 6D). In summary, we find that the stochastic network exhibits comparable performance to the deterministic network. Thus, SGs are suitable for training stochastic SNNs.

To verify that this finding generalizes to a more complex task, we trained a convolutional spiking neural networks (ConvSNN) with three hidden layers on the Spiking Heidelberg Digits (SHD) data set (Cramer et al., 2022) with maximum-over-time readout (see section 8.4.2 and Figure 7A). The stochastic network learned to solve the task with comparable training

and validation accuracy to the deterministic network (see Figure 7B). Specifically, we found that the stochastic ConvSNN achieved a validation accuracy of $97.2 \pm 1.0\%$, (test $85.1 \pm 1.3\%$), compared to $94.7 \pm 0.8\%$ (test $84.3 \pm 1.0\%$) for the deterministic ConvSNN. Furthermore, in the case of the stochastic SNN, we left the escape noise active during validation and testing. However, one can see that the stochastic network performs equally well when tested in a deterministic environment (see Figure 7D). Conversely, the deterministically trained network does not perform well when evaluated under stochastic spike generation. In contrast to the previous task, we found that the Fano factor remained high during training (see Figure 7C), that is, stochasticity is preserved. This is also reflected in a high trial-to-trial variability as shown in Figure 7E. One can see that the spiking activity for three example neurons shows a high variability across trials for the same input. Thus, even for a more complex task, SG descent is well suited for training stochastic SNNs, and stochasticity is preserved after training.

7 Discussion

We have analyzed the theoretical foundations of the widely used SG descent method for multilayer SNNs and shown that SGs can be derived for stochastic networks from the stochAD framework (Arya et al., 2022). The stochAD framework provides a principled way to use AD in discrete stochastic systems, directly applicable to SG descent in stochastic SNNs. Our analysis is based on smoothed stochastic derivatives introduced within the stochAD framework and stipulates that the SD should match the derivative of the escape noise function of the stochastic neuron model. We confirmed in simulations that SG descent allows training well-performing stochastic SNNs with substantial variability comparable to neurobiology. Our work shows that SGs enjoy better theoretical support in stochastic SNNs, which removes some ambiguity of choice of the SD and also sheds light on deterministic SGs as a special case.

One of our key findings is the relation between the functional shape of the escape noise in stochastic neuron models and the SD. This relation is evident in single neurons, where the solution found by SGs and stochAD is equal to the exact derivative of the expected output. This relation has also been described in previous work on shallow networks (Pfister et al., 2006; Brea et al., 2013), where the derivative of the escape noise directly shows up when computing exact derivatives in expectation. In this article, we found that the relation extends to multilayer SNNs when applying stochAD as a theoretical framework. The link between the SD shape and escape noise provides a rigorous mathematical justification for a particular SD choice, which in previous work was typically chosen heuristically.

7.1 Related Work. The issue of nonexistent derivatives is not unique to SNNs but is a well-known challenge when dealing with discrete random

variables and their derivatives. It hence arises in various contexts, such as in general binary neural networks (Courbariaux et al., 2016), sigmoid belief nets (Neal, 1992), or when dealing with categorical distributions (Maddison et al., 2017; Jang et al., 2016). To address this challenge in arbitrary functions, including discrete random variables, the score function estimator, also called REINFORCE (Williams, 1992), is often applied because it provides unbiased estimates. This estimator can be computed through stochastic computation graphs (Schulman et al., 2015). While Arya et al. (2022) use coupled randomness to reduce variance, there are also methods building on reinforcement learning for low-variance gradient estimation (Weber et al., 2019).

Conversely, to address the challenge of BP through nondifferentiable functions in neural networks instead, a commonly used solution is the STE (Hinton, 2012; Bengio et al., 2013), which replaces the derivative of a stochastic threshold function with one, also called the identity STE (Yin et al., 2018). This solution relates closely to SGs used in SNNs. However, SGs are commonly used in deterministic networks and comprise a nonlinear SD like the derivative of a scaled fast sigmoid (Neftci et al., 2019). While both are successful in practice, we still lack a complete theoretical understanding. Several studies have analyzed the theoretical aspects of the STE, looking at the problem from various angles. However, they have each analyzed different versions of the STE, and a complete picture of the underlying theory is still missing. Although none of them analyzed STEs in SNNs (i.e., SGs), examining their results helps to put SGs into perspective. Bengio et al. (2013) originally introduced the identity STE as a biased estimator, saying that it guarantees the correct sign only in shallow networks. Yin et al. (2018) studied coarse gradients, including the STE. They studied this either in a network with only one nonlinearity or in an activation-quantized network with a quantized rectified linear unit (ReLU) as opposed to a Heaviside activation function, as we have in the case of SNNs. They analyzed training stability and investigated which choice of STEs leads to useful weight updates. They concluded that the identity STE does not, while the ReLU and clipped ReLU versions do. They further found that the identity STE might be repelled from certain minima. Liu et al. (2023) showed that the identity STE provides a first-order approximation to the gradient, which was previously shown by Tokui and Sato (2017) specifically for Bernoulli random variables. However, other STEs were not considered, for example, the sigmoidal STE, which would correspond to our SG. Another approach to dealing with discrete distributions was pursued by Jang et al. (2016) and Maddison et al. (2017). They proposed a solution similar to the reparameterization trick but combined with a smooth relaxation of the categorical distribution. This relaxation amounts to training a network with continuous representations by sampling from a Gumbel-Softmax. At the same time, after training, the temperature of the Gumbel-Softmax distribution is set to zero to obtain a discrete output. Hence, while providing discrete output after training,

such a network is trained with continuous neuronal outputs rather than spikes.

Interestingly, most of these solutions deal with discrete functions in the stochastic case, such as stochastic binary perceptrons or Bernoulli random variables, where noise is used for smoothing. But as we saw above, probabilistic approaches in the context of SPMs cannot be applied to multilayer SNNs and BP without approximations. In the past, stochastic SNN models have been implemented mainly for theoretical and biological plausibility reasons (Pfister et al., 2006; Brea et al., 2013). For example Brea et al. (2013) showed a link between the escape noise and the voltage-triplet rule (Clopath & Gerstner, 2010) in shallow networks. Furthermore, there were also approaches that enabled the training of multilayer stochastic SNNs with AD, such as the MultilayerSpiker proposed by Gardner et al. (2015), which uses the spike train itself as the SD of a spike train. Today, SNNs are mostly implemented as networks of deterministic LIF neurons with a Heaviside function as spiking nonlinearity, as they tend to have higher performance. Recently, however, Ma et al. (2023) found empirically that stochastic SNNs with different types of escape noise can be trained with SG descent to high performance. While they showed a connection to deterministic SG descent, they did not discuss the implications for multilayer networks with stateful LIF neurons. Again, the effectiveness of SG descent in stochastic SNNs was confirmed, as suggested by the connection to the stochAD framework found here. Thus, our work not only provides empirical confirmation of the effectiveness of SGs in training stochastic SNNs but also a comprehensive theoretical explanation for their success in multilayer networks.

7.2 Limitations. To gain analytical insight into the theory behind SGs, we had to make some simplifying assumptions. For example, we performed most of our theoretical analysis on perceptrons, which, unlike LIF neurons, have no memory. Nevertheless, our results readily transfer to LIF neurons in discrete time, as such networks can be understood as a special type of binary recurrent neural networks (Neftci et al., 2019) where the output is given by a Heaviside step function. However, one difference is that unlike perceptrons, LIF models have a reset after the spike, which should also be considered when calculating gradients. Smoothed stochastic derivatives allow and suggest backpropagating through the reset mechanism, but previous work has shown that it is beneficial to exclude the reset mechanism from the backward pass, as this leads to more robust performance, especially when the SG is not scaled to one (Zenke & Ganguli, 2018; Zenke & Vogels, 2021; Eshraghian et al., 2023). Both options are possible when using SG-descent, and we have not noticed much difference in performance when scaling the SDs by $\frac{1}{\beta_{SG}}$ to one. However, omitting this scaling negatively affects performance when backpropagating through the reset (see supplementary Figure S2C and compare the asymptotic SuperSpike in Zenke &

Ganguli, 2018). While omitting this scaling can potentially be counteracted by an optimizer using per parameter learning rates, the prefactor due to β_{SG} in the unscaled case can become quite large as it accumulates across layers and thus still affects performance.

Another limitation is that we derived SGs for stochastic SNN models, while they are most commonly used in deterministic networks. This discrepancy may in fact point at a possible advantage. Our simulations suggested that stochastic networks generalize equally well or better than their deterministic counterparts. This difference may be due to stochastic path selection, which leads to lower selection bias compared to the deterministic case. Moreover, when employing small weight updates, each update follows a different path in the tree of spike patterns, effectively averaging the updates over multiple trials, albeit with slightly different parameter sets. Such stochastic averaging helps to reduce the bias present in the smoothed stochastic derivatives. Alternatively, the difference may be due to escape noise acting as an implicit regularizer reminiscent of dropout that promotes better generalization.

Next, while our work motivates a clear link between the SD and a stochastic neuron model with escape noise, it does not say which escape noise to use. This lack may seem like a weakness that merely moves the ambiguity of choice to a different component of the neuron model. Instead, doing so reduces two heuristic choices, namely the choice of escape noise and the shape of the SD, to one. Furthermore, the escape noise function can be constrained using biological data (Jolivet et al., 2006; Pozzorini et al., 2015).

Moreover, the tasks we used to study stochastic SNNs were limited. Historically, stochastic SNNs have been evaluated on tasks that require exact deterministic spike trains at readout such as in the spike train matching task (Pfister et al., 2006; Brea et al., 2013; Gardner et al., 2015). However, despite being performed with stochastic SNNs, such tasks actually punish stochasticity, at least in the readout. Therefore, stochastic SNNs respond by becoming more deterministic during training as we have observed when monitoring the Fano factor (see Figure 6). In our setting, neurons have the possibility to change the effective steepness of the escape noise function by increasing their weights, as it is dependent on $\beta_N \cdot |w|$, and thus get more deterministic. Therefore, we believe that tasks that do not punish stochasticity, such as classification tasks, are much more suitable for evaluating the performance of stochastic SNNs as they allow the training of well-performing SNNs that still show substantial variability in their responses.

Finally, our analysis of the deterministic SGs was also subject to some limitations. In particular, a comparison with the actual gradient, which is either zero or undefined, would not have worked. Instead, we analyzed the deviation of the SG from the actual gradient in differentiable networks that are nonspiking. Doing so, we have shown that SGs in deterministic networks with well-defined gradients deviate from the actual gradients.

In the limiting case $\beta_f \rightarrow \infty$, these networks are equivalent to perceptron networks with step activation functions. Thus, although we could not perform our analysis directly on deterministic SNNs, we consider the above approach to be a good approximation since it preserves the main properties of SGs.

7.3 Future Research. The work in this article motivates several exciting directions for future research. First, we found that although necessary for training, SGs are generally not gradients of a surrogate loss and thus not guaranteed to find a local minimum. Specifically, when replacing the nonexistent derivative of a Heaviside with an SD, the resulting SG is generally not a gradient. It seems unlikely that choosing a different escape noise or SD will alter this result. Thus, in situations requiring stronger theoretical guarantees as those bestowed by gradients, an alternative approach would be to start from a surrogate loss function. While this approach would, by design, yield a proper gradient, it would be more akin to optimizing a smoothed version of the problem, as is the case when using the Gumbel-Softmax or the concrete distribution (Jang et al., 2016; Maddison et al., 2017) rather than defining a surrogate for an ill-behaved gradient. However, this approach would also come with the known difficulties for AD. It is not yet clear whether or how they can be overcome by future work.

We also saw that SGs generally decrease the loss, but they do not necessarily find a local minimum of the loss. This discrepancy is due to the deviation from the actual gradient. Yet learning in deterministic SNNs is only possible due to this deviation because the actual gradient is almost always zero. In fact, deviations from the actual gradient are often also desirable even in conventional ANN training. Bias and variance of the gradient estimator can help to generalize (Ghosh et al., 2023). For instance, in ANNs with a rough loss landscape, mollifying, which induces a deviation from the gradient, is successful in practice (Gulcehre et al., 2017). Regularizers or optimizers, which are often used by default, show another advantage of deviations. For example, many optimizers rely on momentum to speed up training or avoid getting caught in bad local minima. Thus, an interesting line of future work is to study what constitutes a good local minimum in the case of SNNs, which types of biases or deviations from the gradient help to find them, and how this relates to SGs.

Another promising starting point for future work is tied to the stochAD framework and questions pertaining to optimization effectiveness. In this study, we linked SG-descent to smoothed stochastic derivatives, which are biased with respect to the nonsmoothed version based on the full stochastic triple. Thus far, the latter can only be computed in forward-mode AD, which is impractical for neural network training, as it comes with an increased computational cost. An intriguing direction for future research is therefore to investigate forward mode AD with the full stochastic triple to determine whether training improvements due to the reduction in bias

would justify the added computational cost. Furthermore, it would be worth exploring whether this computational cost could be reduced, for instance, by using mixed-mode AD, such as in DECOLLE (Kaiser et al., 2020) or online spatiotemporal learning (Bohnstingl et al., 2023; see Zenke & Nefci, 2021, for a review).

Finally, an exciting direction for future research is to use our theoretically motivated methodology for training stochastic SNNs to address biological questions. In neurobiology, many neurons exhibit trial-to-trial variability. However, the role of this variability is not well understood. Here, functional stochastic SNN models will help to further our understanding of its role in the brain. Stochasticity may play possible roles in representational drift (Mico & O'Leary, 2023), efficient coding (Denève & Machens, 2016), and for biological implementations of latent generative models (Hinton et al., 1995; Rombach et al., 2022). Thus, it would be interesting to study how learning influences representational changes in plastic functional stochastic SNNs and whether we can identify specific dynamic signatures that allow drawing conclusions about the underlying learning mechanisms in the brain.

In conclusion, SGs remain a valuable tool for training both deterministic and stochastic SNNs. In this article, we saw that stochAD provides a theoretical backbone to SG learning that naturally extends to stochastic SNNs. Further, it suggests that the choice of the SD should be matched to the derivative of the escape noise. Training such stochastic SNNs is becoming increasingly relevant for applications with noisy data or noisy hardware substrates and is essential for theoretical neuroscience as it opens the door for studying functional SNNs with biologically realistic levels of trial-to-trial variability.

8 Methods

Our perceptron and SNN models were custom-written in Python 3.10.4 using PyTorch (Paszke et al., 2019), Jax (Bradbury et al., 2018) and the Stok Library for training SNNs (Rossbroich et al., 2022, <https://github.com/fmi-basel/stork>). The code repository can be found at <https://github.com/fmi-basel/surrogate-gradient-theory>. All models were implemented in discrete time. The following sections provide more details on the different neuron models before providing all the necessary information on the two learning tasks, including the architecture and parameters used for a given task.

8.1 Neuron Models.

8.1.1 Perceptron. The perceptron is a simplified version of the LIF neuron model, without memory or reset. The membrane dynamics U_i^l for perceptron i in layer l are given for the special case of a single perceptron by equation 3.1 in the theoretical results section or in general by

$$U_i^l = \sum_j w_{ij} S_j + b_i,$$

where w_{ij} are the feedforward weights, S_j are the binary outputs (spikes) of the previous layer, and b_i is an optional bias term.

8.1.2 Leaky Integrate-and-Fire Neuron. We used an LIF neuron model with exponential current-based synapses (Gerstner et al., 2014). In discrete time, the membrane potential $U_i^l[n]$ of neuron i in layer l at time step n is given by

$$U_i^l[n+1] = (\lambda_{mem} \cdot U_i^l[n] + (1 - \lambda_{mem}) \cdot I_i^l[n]) \cdot (1 - S_i^l[n]), \quad (8.1)$$

where $I_i^l[n]$ is the input current and $S_i^l[n]$ the output spike of the neuron itself, which governs reset dynamics. With multiplicative reset as above, the neuron stays silent for one time step after each spike. The membrane decay variable $\lambda_{mem} = \exp\left(-\frac{\Delta t}{\tau_{mem}}\right)$ is defined through the membrane time constant τ_{mem} and the chosen time step Δt . The input current $I_i^l[n]$ is given by

$$I_i^l[n+1] = \lambda_{syn} \cdot I_i^l[n] + \underbrace{\sum_j w_{ij}^l \cdot S_j^{l-1}[n]}_{\text{feedforward}} + \underbrace{\sum_k v_{ik}^l \cdot S_k^l[n]}_{\text{recurrent}}, \quad (8.2)$$

where w_{ij} and v_{ik} are the feedforward and recurrent synaptic weights, respectively, corresponding to the previous layers' spikes S_j^{l-1} and the same layers' spikes S_k^l , respectively. The synaptic decay variable is again given as $\lambda_{syn} = \exp\left(-\frac{\Delta t}{\tau_{syn}}\right)$, defined through the synaptic time constant τ_{syn} . At the beginning of each minibatch, the initial membrane potential value of all neurons was set to their resting potential $U_i^l[0] = U_{rest} = 0$, and the initial value for the synaptic current was zero as well, $I_i^l[0] = 0$.

8.2 Spike Generation. The generation of a spike follows the same mechanism in both the LIF and the perceptron neuron model. Depending on the membrane potential value, the activation function generates a spike or not. The following paragraphs highlight the differences between the deterministic and the stochastic cases.

8.2.1 Deterministic Spike Generation. A spike is generated in every time step, in which the membrane potential crosses a threshold θ . Hence, the Heaviside step function $H(\cdot)$ is used to generate a spike deterministically at time step n :

$$S_i^l[n] = H(U_i^l[n] - \theta).$$

8.2.2 Stochastic Spike Generation. A stochastic neuron with escape noise (Gerstner et al., 2014) may spike even if the membrane potential is below the

threshold, or vice versa not spike, even if the membrane potential is above the threshold. Therefore, in discrete time, the probability of spiking in time step n is

$$p_i^l[n] = f(U_i^l[n] - \theta) = \sigma_{\beta_N}(U_i^l[n] - \theta) \quad (8.3)$$

for each neuron. We used $\sigma_{\beta_N}(x) = \frac{1}{1+\exp(-\beta_N \cdot x)}$, if not stated otherwise. The hyperparameter β_N defines the steepness of the sigmoid, with $\beta_N \rightarrow \infty$ leading to deterministic spiking. Spikes were then drawn from a Bernoulli distribution with probability $p_i^l[n]$, hence,

$$S_i^l[n] \sim \text{Ber}(p_i^l[n]). \quad (8.4)$$

The expected value of spiking equals the spike probability, so $\mathbb{E}[S_i^l[n]] = p_i^l[n]$.

To emphasize the similarity to deterministic spike generation, we can consider an equivalent formulation, where the stochasticity is absorbed into the threshold such that equations 8.3 and 8.4 are combined into

$$S_i^l[n] = H(U_i^l[n] - \Theta). \quad (8.5)$$

The threshold $\Theta = \theta + \xi$ is defined by its mean θ , the deterministic threshold, and a zero-mean noise ξ . For equivalence to the version with the Bernoulli random variable (see equation 8.4) the noise ξ is distributed as the $\text{logit}(x) = \log\left(\frac{x}{1-x}\right)$ where x is drawn from a uniform distribution over the interval $[0, 1]$, that is, $x \sim \mathcal{U}(0, 1)$.

8.3 Differentiable Example Network. The minimal example network (see Figure 4A) was simulated using Jax (Bradbury et al., 2018). To implement the main essence of SG descent in a differentiable network, we constructed a network of perceptrons with sigmoid activation functions (see equation 4.1). The SG was implemented by a less steep sigmoid, which was used instead of the actual activation function on the backward pass.

8.3.1 Integrated (Surrogate) Gradients. In general, the integral of the derivative of the loss should equal again the loss $\int \frac{\partial \mathcal{L}}{\partial w} dw = \mathcal{L}$. The same holds true for the network output. In Figure 4, we computed this quantity for $w \in [-0.2, 0.2]$, as well as in two dimensions for the parameters v_1 and v_2 . In Figure 5, we did not compute this along a line but instead chose to compute this along a closed path in parameter space—a circle. To do so, we integrated the SG along a circle in a two-dimensional hyperplane, spanned by two randomly chosen orthonormal vectors d_1 and d_2 in parameter space. The position along the circle is defined by an angle α and the circle is parameterized by a and b such that $a = r \cdot \sin(\alpha)$ and $b = r \cdot \cos(\alpha)$ with r the

Table 2: Noise and Surrogate Function Parameters.

	Spike-train matching task		Classification task	
	stochastic	deterministic	stochastic	deterministic
Number of trials	10	1	1	1
Learning rate	$\eta_{\text{hid}} = 10^{-05}$ $\eta_{\text{out}} = 10^{-05}$	$\eta_{\text{hid}} = 10^{-05}$ $\eta_{\text{out}} = 10^{-04}$	0.01	0.01
<i>Escape noise</i>				
Function	$\sigma(\cdot)$	step	$\sigma(\cdot)$	step
Parameter	$\beta_{\text{hid}} = 10$ $\beta_{\text{out}} = 100$	–	$\beta = 10$	–
<i>Surrogate gradient</i>				
Function	$\sigma'(\cdot)$	SuperSpike	SuperSpike	SuperSpike
Parameter	$\beta_{\text{hid}} = 10$	$\beta = 10$	$\beta = 10$	$\beta = 10$

Notes: Parameters used in our numerical simulations for feedforward networks on the spike-train matching task and multi-layer recurrent ConvSNNs on the SHD classification task. We selected the learning rate based on the best validation accuracy (see supplementary Figure S2B). The SuperSpike nonlinearity $h(x)$ is the derivative of a fast sigmoid scaled by $\frac{1}{\beta}$: $h(x) = \frac{1}{(\beta|x|+1)^2}$.

radius of the circle. Hence, when integrating along the circle, the weights θ in the network changed according to the angle α ,

$$\theta_{\alpha} = d_1 \cdot a(\alpha) + d_2 \cdot b(\alpha).$$

8.4 Learning Tasks. We trained SNNs to evaluate the differences in learning between the stochastic and deterministic SNN models trained with SG-descent. For this purpose, we chose a spike train matching and a classification task to cover different output modalities.

8.4.1 Spike Train Matching. If an SNN can learn a precise timing of spikes, it must be able to match its output spike times with some target output spike times. Therefore, in the spike train matching task, we ask whether both the stochastic and the deterministic network can learn deterministic output spike trains. For the training, no optimizer is used, and since we use a minibatch size of one, this means we apply batch gradient descent. The details about the used architecture, neuronal, and training parameters are given in the following paragraphs as well as in Tables 2, 3, and 4 for both, the stochastic and the deterministic versions of the network. The code was written in Jax.

Task. The target spike trains to match were generated from a picture of the Radcliffe Camera in Oxford using dithering to create a binary image. We set this image as our target spike raster, where the x -axis corresponds to time steps and the y -axis is the neuron index. Hence, the image provides a binary spike train for each readout neuron. As an input, we used frozen

Table 3: Architecture Parameters.

	Spike-train matching task	Classification task
Data Set	Radcliffe Camera	SHD
No. input neurons	200	700
No. hidden neurons	200	16-32-64
No. output neurons	200	20
No. training epochs	5000	200
Time step	1 ms	2 ms
Duration	198 ms	700 ms
Mini-batch size	1	400
Kernel size (ff)	–	21-7-7
Stride (ff)	–	10-3-3
Padding (ff)	–	0-0-0
Kernel size (rec)	–	5
Stride (rec)	–	1
Padding (rec)	–	2

Note: Parameters used in numerical simulations for feedforward (ff) SNNs on the spike-train matching task and multilayer recurrent (rec) ConvSNNs on the SHD classification task.

Table 4: Neuronal and Training Parameters.

	Spike-train matching task	Classification task
Loss	L_2 or van Rossum distance	Maximum over time
Optimizer	None (gradient descent)	SMORMS3
<i>Neuronal parameters</i>		
Spike threshold	1	1
Resting potential	0	0
τ_{mem}	10 ms	20 ms
τ_{syn}	5 ms	10 ms
$\tau_{\text{mem}}^{\text{RO}}$	–	700 ms
Reset	at same time step	at next time step
<i>Activity regularizer</i>		
ϑ_{upper}	–	7
λ_{upper}	–	0.01

Note: Parameters used in numerical simulations for feedforward SNNs on the spike-train matching task and deep recurrent ConvSNNs on the SHD classification task.

Poisson-distributed spike trains with a per neuron firing rate of 50 Hz. This created a one-image data set that requires 200 input neurons and 200 output neurons and has a duration of 198 time steps, that is, 198 ms when using $\Delta t = 1$ ms.

Network architecture. The spike train matching task is an easy task that could also be solved by an SNN without a hidden layer. However, since we

knew that in the shallow case without a hidden layer, the only difference between our stochastic and our deterministic versions would lie in the loss, we decided to use a network with one hidden layer. Hence, the network architecture we used was a feedforward network with 200 input units, 200 hidden units, and 200 readout units, run with $\Delta t = 1$ ms for a total of 198 time steps (see also Table 3). Weights were initialized in the fluctuation-driven regime (Rossbroich et al., 2022) with a mean of zero and target membrane fluctuations $\sigma_U = 1$. For the stochastic networks, we averaged the update over 10 trials: we sampled 10 paths in the tree of spike patterns, used each of them to compute an SG, and took their average to perform the weight update before sampling another path for the next weight update (see Figure 3).

Reset at same time step. Matching a target binary image without further constraints with a spike train might require a neuron to spike in two adjacent time steps. However, this is not possible with the membrane dynamics as in equation 8.1, where after every spike, the neuron stays silent for one time step. Hence for this task, we used slightly modified membrane dynamics,

$$U_i^l[n+1] = \lambda_{mem} \cdot U_i^l[n] \cdot (1 - S_i^l[n]) + (1 - \lambda_{mem}) \cdot I_i^l[n],$$

where the reset was applied at the same time step as the spike and thus high enough input in the next time step could make the neuron fire again.

Loss functions. We trained the network using an L_2 loss function,

$$L_2 = \frac{1}{N} \sum_{i=1}^{M^L} \sum_{n=1}^T \left(S_i^L[n] - \hat{S}_i[n] \right)^2,$$

to compute the distance between target spike train $\hat{S}_i$ for readout neuron i and output spike train $S_i^L[n]$ at the readout layer L , where M^L is the number of readout neurons and T is the number of time steps. Furthermore, we also monitor the van Rossum distance (van Rossum, 2001),

$$L_{vR} = \frac{1}{2} \int_{-\infty}^{\infty} \left((\alpha \hat{S}_i - \alpha S_i)(s) \right)^2 ds,$$

between the target spike train and the output spike train, which might be a better distance for spike trains, since it also takes temporal structure into account by punishing a spike that is five time steps off more than a spike that is only one time step off. It does so by convolving the output and target spike train first with a temporal kernel α , before computing the squared

distance. We chose α to be equivalent to the ϵ -kernel of the LIF neuron,

$$\alpha(t) = \frac{1}{1 - \frac{\tau_{\text{mem}}}{\tau_{\text{syn}}}} \left(\exp\left(-\frac{t}{\tau_{\text{syn}}}\right) - \exp\left(-\frac{t}{\tau_{\text{mem}}}\right) \right),$$

with $\tau_{\text{mem}} = 10$ ms, $\tau_{\text{syn}} = 5$ ms.

Fano factor. The Fano factor F was calculated on the hidden layer and output spike trains S as $F = \frac{\sigma_S^2}{\mu_S}$.

8.4.2 Classification on SHD. To evaluate performance differences, we also evaluated both the stochastic and the deterministic version on a classification task using a multilayer recurrent ConvSNN (as in Rossbroich et al., 2022), hence also increasing the difficulty for the stochastic model, which now had to cope with multiple noisy layers. The details about the used architecture, neuronal, and training parameters are given in the following paragraphs as well as in Tables 2, 3, and 4 for both, the stochastic and the deterministic versions of the network.

Task. For the classification task, we used the SHD data set (Cramer et al., 2022), a real-world auditory data set containing spoken digits in German and English from different speakers; hence, it has 20 classes. The data set can be downloaded from <https://iee-dataport.org/open-access/heidelberg-spiking-datasets>. Cramer et al. (2022) preprocessed the recordings using a biologically inspired cochlea model to create input spike trains for $n = 700$ neurons. The duration of the samples varies; hence, we decided to consider only the first $T_{\text{SHD}} = 700$ ms of each sample, which covers more than 98% of the original spikes. For our numerical simulations, we binned the spike trains using a $\Delta t = 2$ ms and hence had $\frac{\Delta t}{T_{\text{SHD}}} = 350$ time steps in this task. Ten percent of the training data was used as a validation set, and for testing, we used the officially provided test set, which contains only speakers that did not appear in the training set.

Network architecture. We used a multilayer recurrent ConvSNN for the SHD classification task (see also Table 3). There were 700 input neurons that directly got fed the input spikes from the SHD data set. The network had three recurrently connected hidden layers and used one-dimensional convolution kernels. Weights were initialized in the fluctuation-driven regime with a mean of zero, target membrane fluctuations $\sigma_U = 1$, and the proportion of fluctuations due to the feedforward input was $\alpha = 0.9$. The network size as well as the parameters for the feedforward and recurrent convolutional operations are summarized in Table 3. We performed only one trial per update for the stochastic case.

Readout units. As opposed to the previous task, a classification task requires special readout units. The readout units did have the same membrane and current dynamics as a normal LIF neuron (see equations 8.1 and 8.2), but they did not spike. Furthermore, we used a different membrane

time constant $\tau_{\text{mem}}^{\text{RO}} = T_{\text{SHD}} = 700$ ms for the readout units. This allowed us to read out their membrane potential for classification (see the paragraph below on loss function).

Activity regularization. To prevent tonic firing, we used activity regularization to constrain the upper bound of firing activity. To this end, we constructed an additional loss term as a soft upper bound on the average firing activity of each feature in our convolutional layers. Hence for every layer, we computed a term,

$$g_{\text{upper}}^{l,k} = \left(\left[\frac{1}{M^l} \sum_i^{M^l} \zeta_i^{l,k} - \vartheta_{\text{upper}} \right]_+ \right)^2, \quad (8.6)$$

where $M^l = n_{\text{features}} \times n_{\text{channels}}$ is the number of neurons in layer l and $\zeta_i^{l,k} = \sum_n^T S_i^{l,k}[n]$ is the spike count of neuron i in layer l given input k . We chose the parameter $\vartheta_{\text{upper}} = 7$ to constrain the average firing rate to 10 Hz. Hence, the total upper-bound activity regularization loss is given by

$$L_{\text{upper}} = -\lambda_{\text{upper}} \sum_{l=1}^L \sum_{f=1}^{F^l} g_{\text{upper}}^{l,k}, \quad (8.7)$$

where $\lambda_{\text{upper}} = 0.01$ was the regularization strength. Those parameters can also be found in Table 4.

Loss function. As this is a classification task, we used a maximum-over-time readout. To that end, we used a standard cross-entropy loss,

$$L_{\text{CE}} = -\frac{1}{K} \sum_{i=1}^K \sum_{c=1}^C y_c^k \log(p_c^k), \quad (8.8)$$

to sum over all samples K and all classes C . The correct class is encoded in y_c^k as a one-hot encoded target for the input k . To compute the single probabilities p_c^k for each class c , we first read out the maximum membrane potential values of each readout neuron over simulation time to get the activities,

$$a_c^k = \max_n (U_c^L[n]). \quad (8.9)$$

From those activities, we computed the probabilities using a softmax function $p_c^k = \frac{\exp(a_c^k)}{\sum_{c'} \exp(a_{c'}^k)}$.

Optimizer. We used the squared mean over root mean squared cubed (SMORMS3) optimizer (Funk, 2015; Rossbroich et al., 2022), which chooses

a learning rate based on how noisy the gradient is. The SMORMS3 optimizer keeps track of the three values g , g_2 , and m , which were initialized to $g = g_2 = 0$ and $m = 1$. They are updated after every minibatch as follows:

$$\begin{aligned} r &= \frac{1}{m+1}. \\ g &= (1-r) \cdot g + r \cdot \left(\frac{\partial \mathcal{L}}{\partial \theta} \right). \\ g_2 &= (1-r) \cdot g_2 + r \cdot \left(\frac{\partial \mathcal{L}}{\partial \theta} \right)^2. \\ m &= 1 + m \cdot \frac{1-g^2}{g_2 + \epsilon}. \end{aligned}$$

Given this, SMORMS3 computes the current effective learning rate as $\eta_{\text{current}} = \frac{\min\left(\eta, \frac{g^2}{g_2 + \epsilon}\right)}{\sqrt{g_2 + \epsilon}}$, where η is the initially chosen learning rate and ϵ is a small constant to avoid division by zero. Therefore, the parameter update is $\Delta\theta = -\eta_{\text{current}} \cdot \frac{\partial \mathcal{L}}{\partial \theta}$.

Fano factor. The Fano factor was calculated after taking a moving average over the spike train S with a window of 10 time steps (20 ms) to get S_{bin} . Subsequently, the Fano factor F was computed as

$$F = \frac{\sigma_{S_{\text{bin}}}^2}{\mu_{S_{\text{bin}}}}.$$

Acknowledgments

We thank Frank Schäfer, Guillaume Bellec, and Wulfram Gerstner for stimulating discussions. This work was supported by the Swiss National Science Foundation (grant PCEFP3_202981) and the Novartis Research Foundation.

References

- Arya, G., Schauer, M., Schäfer, F., & Rackauckas, C. (2022). Automatic differentiation of programs with discrete randomness. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *Advances in neural information processing systems*, 35, 10435–10447.
- Bellec, G., Salaj, D., Subramoney, A., Legenstein, R., & Maass, W. (2018). Long short-term memory and learning-to-learn in networks of spiking neurons. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, & R. Garnett (Eds.), *Advances in neural information processing systems*, 31. Curran.
- Bengio, Y., Léonard, N., & Courville, A. (2013). *Estimating or propagating gradients through stochastic neurons for conditional computation*. arXiv:1308.3432.

- Bohnstingl, T., Woźniak, S., Pantazi, A., & Eleftheriou, E. (2023). Online spatiotemporal learning in deep neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 34(11), 8894–8908. 10.1109/TNNLS.2022.3153985
- Bohte, S. M. (2011). Error-backpropagation in networks of fractionally predictive spiking neurons. In T. Honkela, W. Duch, M. Girolami, & S. Kaski (Eds.), *Artificial neural networks and machine learning—ICANN 2011* (pp. 60–68). Springer. 10.1007/978-3-642-21735-7_8
- Bohte, S. M., Kok, J. N., & La Poutré, H. (2002). Error-backpropagation in temporally encoded networks of spiking neurons. *Neurocomputing*, 48(1–4), 17–37. 10.1016/S0925-2312(01)00658-0
- Bradbury, J., Frosting, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., ... Zhang, Q. (2018). *JAX: Composable transformation of Python+NumPy programs*.
- Brea, J., Senn, W., & Pfister, J.-P. (2013). Matching recall and storage in sequence learning with spiking neural networks. *Journal of Neuroscience*, 33(23), 9565–9575. 10.1523/jneurosci.4098-12.2013
- Clopath, C., & Gerstner, W. (2010). *Voltage and spike timing interact in STDP: A unified model*. <https://www.frontiersin.org/journals/synaptic-neuroscience/articles/10.3389/fnsyn.2010.00025/full>
- Courbariaux, M., Hubara, I., Soudry, D., El-Yaniv, R., & Bengio, Y. (2016). *Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1*. <http://arxiv.org/abs/1602.02830>
- Cramer, B., Stradmann, Y., Schemmel, J., & Zenke, F. (2022). The Heidelberg spiking data sets for the systematic evaluation of spiking neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 33(7), 2744–2757. 10.1109/TNNLS.2020.3044364
- Denève, S., & Machens, C. K. (2016). Efficient codes and balanced networks. *Nature Neuroscience*, 19(3), 375–382. 10.1038/nn.4243
- Eshraghian, J. K., Ward, M., Neftci, E., Wang, X., Lenz, G., Dwivedi, G., ... Lu, W. D. (2023). Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, 111(9), 1016–1054. 10.1109/jproc.2023.3308088
- Esser, S. K., Merolla, P. A., Arthur, J. V., Cassidy, A. S., Appuswamy, R., Andreopoulos, A., ... Modha, D. S. (2016). Convolutional networks for fast, energy-efficient neuromorphic computing. *Proceedings of the National Academy of Sciences of the United States of America*, 113(41), 11441–11446. 10.1073/pnas.1604850113
- Fu, M. C. (2006). Gradient estimation. In S. G. Henderson & B. L. Nelson (Eds.), *Handbooks in operations research and management science* (pp. 575–616). Elsevier. 10.1016/S0927-0507(06)13019-4
- Funk, S. (2015). *RMSprop loses to SMORMS3: Beware the epsilon!* <https://sifter.org/simon/journal/20150420.html>
- Gardner, B., Sporea, I., & Grüning, A. (2015). Learning spatiotemporally encoded pattern transformations in structured spiking neural networks. *Neural Computation*, 27(12), 2548–2586. 10.1162/NECO_a_00790
- Gerstner, W., & Kistler, W. M. (2002). *Spiking neuron models*. Cambridge University Press.
- Gerstner, W., Kistler, W. M., Naud, R., & Paninski, L. (2014). *Neuronal dynamics: From single neurons to networks and models of cognition*. Cambridge University Press.

- Ghosh, A., Liu, Y. H., Lajoie, G., Kording, K., & Richards, B. A. (2023). How gradient estimator variance and bias impact learning in neural networks. In *Proceedings of the International Conference on Learning Representations*. <https://openreview.net/forum?id=EBC60mxBwyw>
- Glasserman, P. (2003). Estimating sensitivities. In P. Glasserman (Ed.), *Monte Carlo methods in financial engineering* (pp. 377–420). Springer. 10.1007/978-0-387-21617-1_7
- Gulcehre, C., Moczulski, M., Visin, F., & Bengio, Y. (2017). Mollifying networks. In *Proceedings of the International Conference on Learning Representations*. <https://openreview.net/forum?id=r1G4z8cge>
- Gütig, R., & Sompolinsky, H. (2006). The tempotron: A neuron that learns spike timing–based decisions. *Nature Neuroscience*, 9(3), 420–428. 10.1038/nn1643
- Hammouamri, I., Khalfaoui-Hassani, I., & Masquelier, T. (2023). *Learning delays in spiking neural networks using dilated convolutions with learnable spacings*. arXiv:2306.17670.
- Herranz-Celotti, L., & Rouat, J. (2024). *Stabilizing spiking neuron training*. arXiv:2202.00282v4.
- Hinton, G. E. (2012). *Lectures from the 2012 Coursera course: Neural networks for machine learning*. https://www.cs.toronto.edu/~hinton/coursera_lectures.html
- Hinton, G. E., Dayan, P., Frey, B. J., & Neal, R. M. (1995). The “wake-sleep” algorithm for unsupervised neural networks. *Science*, 268(5214), 1158–1161. 10.1126/science.7761831
- Huh, D., & Sejnowski, T. J. (2018). Gradient descent for spiking neural networks. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, & R. Garnett (Eds.), *Advances in neural information processing systems*, 31. Curran.
- Jang, E., Gu, S., & Poole, B. (2016). Categorical reparameterization with Gumbel-Softmax. In *Proceedings of the International Conference on Learning Representations*.
- Jang, H., Simeone, O., Gardner, B., & Gruning, A. (2019). An introduction to probabilistic spiking neural networks: Probabilistic models, learning rules, and applications. *IEEE Signal Processing Magazine*, 36(6), 64–77. 10.1109/MSP.2019.2935234
- Jolivet, R., Rauch, A., Lüscher, H.-R., & Gerstner, W. (2006). Predicting spike timing of neocortical pyramidal neurons by simple threshold models. *Journal of Computational Neuroscience*, 21(1), 35–49. 10.1007/s10827-006-7074-5
- Kaiser, J., Mostafa, H., & Neftci, E. (2020). Synaptic plasticity dynamics for deep continuous local learning (DECOLLE). *Frontiers in Neuroscience*, 14. 10.3389/fnins.2020.00424
- Klos, C., & Memmesheimer, R.-M. (2023). *Smooth exact gradient descent learning in spiking neural networks*. arXiv:2309.14523.
- Lillicrap, T. P., Santoro, A., Marris, L., Akerman, C. J., & Hinton, G. (2020). Backpropagation and the brain. *Nature Reviews Neuroscience*, 21, 335–1346. 10.1038/s41583-020-0277-3
- Liu, L., Dong, C., Liu, X., Yu, B., & Gao, J. (2023). *Bridging discrete and backpropagation: Straight-through and beyond*. arXiv:2304.08612.
- Ma, G., Yan, R., & Tang, H. (2023). *Exploiting noise as a resource for computation and learning in spiking neural networks*. arXiv:2305.16044.
- Maddison, C. J., Mnih, A., & Teh, Y. W. (2017). *The concrete distribution: A continuous relaxation of discrete random variables*. arXiv:1611.00712.

- Marschall, O., Cho, K., & Savin, C. (2019). *A unified framework of online learning algorithms for training recurrent neural networks*. arXiv:1907.02649. <http://arxiv.org/abs/1907.02649>
- Memmesheimer, R.-M., Rubin, R., Ölveczky, B. P., & Sompolinsky, H. (2014). Learning precisely timed spikes. *Neuron*, 82(4), 925–938. 10.1016/j.neuron.2014.03.026
- Micou, C., & O’Leary, T. (2023). Representational drift as a window into neural and behavioural plasticity. *Current Opinion in Neurobiology*, 81, 102746. 10.1016/j.conb.2023.102746
- Mostafa, H. (2017). Supervised learning based on temporal coding in spiking neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 29(7). 10.1109/TNNLS.2017.2726060
- Neal, R. M. (1992). Connectionist learning of belief networks. *Artificial Intelligence*, 56(1), 71–113. 10.1016/0004-3702(92)90065-6
- Neftci, E. O., Mostafa, H., & Zenke, F. (2019). Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6), 51–63. 10.1109/MSP.2019.2931595
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, & R. Garnett (Eds.), *Advances in neural information processing systems*, 32. Curran.
- Pfister, J.-P., Toyoizumi, T., Barber, D., & Gerstner, W. (2006). Optimal spike-timing-dependent plasticity for precise action potential firing in supervised learning. *Neural Computation*, 18(6), 1318–1348. 10.1162/neco.2006.18.6.1318
- Pozzorini, C., Mensi, S., Hagens, O., Naud, R., Koch, C., & Gerstner, W. (2015). Automated high-throughput characterization of single neurons by means of simplified spiking models. *PLOS Computational Biology*, 11(6), e1004275. 10.1371/journal.pcbi.1004275
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). *High-resolution image synthesis with latent diffusion models*. arXiv:2112.10752.
- Rossbroich, J., Gygax, J., & Zenke, F. (2022). Fluctuation-driven initialization for spiking neural network training. *Neuromorphic Computing and Engineering*, 2(4), 044016. 10.1088/2634-4386/ac97bb
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536. 10.1038/323533a0
- Schulman, J., Heess, N., Weber, T., & Abbeel, P. (2015). Gradient estimation using stochastic computation graphs. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, & R. Garnett (Eds.), *Advances in neural information processing systems*, 28 (pp. 3528–3536). Curran.
- Tokui, S., & Sato, I. (2017). Evaluating the variance of likelihood-ratio gradient estimators. In *Proceedings of the 34th International Conference on Machine Learning* (pp. 3414–3423).
- van Rossum, M. C. W. (2001). A novel spike distance. *Neural Computation*, 13(4), 751–763. 10.1162/089976601300014321
- Weber, T., Heess, N., Buesing, L., & Silver, D. (2019, April). Credit assignment techniques in stochastic computation graphs. In *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics* (pp. 2650–2660).

- Werfel, J., Xie, X., & Seung, H. S. (2004). Learning curves for stochastic gradient descent in linear feedforward networks. In L. Saul, Y. Weiss, & L. Bottou (Eds.), *Advances in neural information processing systems*, 17 (pp. 1197–1204). MIT Press.
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4), 229–256. 10.1007/BF00992696
- Wunderlich, T. C., & Pehle, C. (2021). Event-based backpropagation can compute exact gradients for spiking neural networks. *Scientific Reports*, 11(1), 12829. 10.1038/s41598-021-91786-z
- Yin, P., Lyu, J., Zhang, S., Osher, S., Qi, Y., & Xin, J. (2018). Understanding straight-through estimator in training activation quantized neural nets. In *Proceedings of the International Conference on Learning Representations*.
- Zenke, F., & Ganguli, S. (2018). SuperSpike: Supervised learning in multilayer spiking neural networks. *Neural Computation*, 30(6), 1514–1541. 10.1162/neco_a_01086
- Zenke, F., & Neftci, E. O. (2021). Brain-inspired learning on neuromorphic substrates. *Proceedings of the IEEE*, 109(5), 935–950. 10.1109/JPROC.2020.3045625
- Zenke, F., & Vogels, T. P. (2021). The remarkable robustness of surrogate gradient learning for instilling complex function in spiking neural networks. *Neural Computation*, 33(4), 899–925. 10.1162/neco_a_01367

Received May 6, 2024; accepted December 23, 2024.